

AERO40003 Computing and Numerical Methods 1

Part 2: Numerical Methods

Computing Lab 1

Dr. Roderick Lubbock

Problem 1.1

In a computer, integers need to be stored in a memory location in binary format. The minimum number of bits needed for an integer word in binary to represent all integers between 0 and 1024 (inclusive) is (neglect the need to store the sign in this problem) $2^x = 1024 \therefore x = \log_2 10$

a) 8 b) 9 c) 10 **d) 11**

Problem 1.2

This problem is related to the storage of numbers in a computer. Consider the following simple Matlab script. Type it into your Matlab editor and run it. Explain the output.

```
clear all
clc
t = 0.1;
n = 1:9;
e = n/10 - n*t;
e;
sprintf('%0.20f\t',n/10)
sprintf('%0.20f\t',n*t)
sprintf('%0.20f\t',e)
```

Problem 1.3

Let's type in the code shown here:

```
%% DEFINE VARS in DEFAULT DOUBLE PRECISION
A = (2.0); %other values to try: 2.0; 0.5; 0.2;
cumul_total = (0.0);

%% PERFORM OPERS
for iii = 1:1e6
    cumul_total = cumul_total + A; %add A a million times
end

format long %SHORT IS DEFAULT
cumul_total % This outputs the total at the end
```

% we can also use commands like `sprintf('%0.22f\n', cumul_total_s)` to give us, in this case, 22 decimal places.

What value does `cumul_total` finally takes? Now change the value of `A` to 0.5 and then to 0.2. What is the final value of `cumul_total` now? Explain your findings.

Repeat the exercise, forcing Matlab to use single-precision quantities. By default, Matlab stores variables using double precision storage. As we saw in lectures, this means that in the computer, the binary representations of the decimal values we entered for `A` are truncated to 52 digits of binary or about 16 in decimal. Matlab can be forced to store `A` using single precision, which will truncate the binary representation to 23 digits or about 7 in decimal. Explain your new findings.

Problem 1.4

- Write 2.0, 0.3 and 0.25 first in binary, and then in the format a computer would store them (i.e. using a 1.mantissa $\times 2^c$ format. Note: there is no need in this exercise to express c in binary).
- Computers store numbers in binary. Let's consider you have a very primitive computer whose mantissa (hint: the computer only stores mantissa, i.e. gets rid of the first 1) can store the first 5 digits of a binary number. Truncate your (computer binary format) answers from part a) to 5 mantissa digits, and then convert these truncated numbers back to decimal form. What has happened? Why? What are the implications for the performance of your primitive computer?

$$a) \quad 2.0 = 10_2 = (-1)^0 (10_2)$$

$$0.3 = (0.010011001100110\dots)_2 = 0.01\overline{001}_2 = 0.01\overline{0011}_2$$

$$= (-1)^0 (1 + 0.\overline{0011}_2) \times 2^{-2}$$

$$= (-1)^0 (1 + 0.\overline{0011}_2) \times 2^{125-127} \quad \text{OR} \quad (-1)^0 (1 + 0.\overline{0011}_2) \times 2^{1021-1023}$$

$$0.25 = 0.01_2 = (-1)^0 (1 + 0_2) \times 2^{125-127} \quad \text{OR} \quad (-1)^0 (1 + 0_2) \times 2^{1021-1023}$$

$$b) \quad 2.0 = 10_2 \rightarrow \cancel{00010_2 = 2.0} \quad (-1)^0 (10_2) = 0.2$$

$$0.3 = 0.01\overline{001}_2 \rightarrow \cancel{0.0100_2 = 0.25} \quad (-1)^0 (1 + 0.00110_2) \times 2^{-2} \\ = 0.296875$$

$$0.25 = 0.01_2 \rightarrow \cancel{0.0100_2 = 0.25} \quad (-1)^0 (1 + 0_2) \times 2^{125-127} = 0.25$$

Problem 1.2

```
clear all
clc
t = 0.1;
n = 1:9;
e = n/10 - n*t;
e;
sprintf('%.20f\t',n/10)
sprintf('%.20f\t',n*t)
sprintf('%.20f\t',e)
```

```
ans =
'0.100000000000000000555    0.200000000000000001110    0.29999999999999998890    0.400000000000000002220    0.500000000000000003330
ans =
'0.100000000000000000555    0.200000000000000001110    0.300000000000000004441    0.400000000000000002220    0.500000000000000003330
ans =
'0.000000000000000000000    0.000000000000000000000    -0.000000000000000005551    0.000000000000000000000    0.000000000000000003330
```

The output represent the quantization error (rounding error) when store the number on computer.

The first output represent the error in each term.

Considered the error already exist in 0.1, the second output represent the errors by 0.1.

The third term represent the error between the first and second term.

Problem 1.3

```
%% DEFINE VARS in DEFAULT DOUBLE PRECISION
A = (2.0); %other values to try: 0.5;0.2;
cumul_total = (0.0);
%% PERFORM OPERS
for iii = 1:1e6
    cumul_total = cumul_total + A; %add A a million times
end
format long %SHORT IS DEFAULT
cumul_total % This outputs the total at the end
```

```
cumul_total =
    2000000
```

% we can also use commands like `sprintf('%.22f\n', cumul_total_s)` to give us, in this case, 22

```
%% DEFINE VARS in DEFAULT DOUBLE PRECISION
A = (0.5); %other values to try: 0.2;
cumul_total = (0.0);
%% PERFORM OPERS
for iii = 1:1e6
    cumul_total = cumul_total + A; %add A a million times
end
format long %SHORT IS DEFAULT
```

```
cumul_total % This outputs the total at the end
```

```
cumul_total =  
    500000
```

```
%% DEFINE VARS in DEFAULT DOUBLE PRECISION  
A = (0.2);  
cumul_total = (0.0);  
%% PERFORM OPERS  
for iii = 1:1e6  
    cumul_total = cumul_total + A; %add A a million times  
end  
format long %SHORT IS DEFAULT  
cumul_total % This outputs the total at the end
```

```
cumul_total =  
    2.000000000026658e+05
```

Cumul finally takes 2000000 when A=2.0, takes 500000 when A=0.5 and takes 2.000000000026658e+05 when A=0.2 when store A using double precision.

```
%% DEFINE VARS in DEFAULT SINGLE PRECISION  
A = single(2.0);  
cumul_total = single(0.0);  
%% PERFORM OPERS  
for iii = 1:1e6  
    cumul_total = cumul_total + A; %add A a million times  
end  
format long %SHORT IS DEFAULT  
cumul_total % This outputs the total at the end
```

```
cumul_total = single  
    2000000
```

```
%% DEFINE VARS in DEFAULT SINGLE PRECISION  
A = single(0.5);  
cumul_total = single(0.0);  
%% PERFORM OPERS  
for iii = 1:1e6  
    cumul_total = cumul_total + A; %add A a million times  
end  
format long %SHORT IS DEFAULT  
cumul_total % This outputs the total at the end
```

```
cumul_total = single  
    500000
```

```
%% DEFINE VARS in DEFAULT SINGLE PRECISION  
A = single(0.2);
```

```
cumul_total = single(0.0);  
%% PERFORM OPERS  
for iii = 1:1e6  
    cumul_total = cumul_total + A; %add A a million times  
end  
format long %SHORT IS DEFAULT  
cumul_total % This outputs the total at the end
```

```
cumul_total = single  
2.0191669e+05
```

Cumul finally takes 2000000 when A=2.0, takes 500000 when A=0.5 but takes 2.0191669e+05 when A=0.2 when store A using single precision.

There would be a large difference when using single precision when A is not multiple of 2.